minicut 動画編集エディタ

組み込み LuaJIT & プラグイン API リファレンスマニュアル
LuaJIT 5.1 ランタイム ドッキング可能ツールパネル ツールシェルフ・ドック ネットワークライブラリ (libcurl) FreeType テキスト生成

1. はじめに & アーキテクチャ

minicut は高速な組み込み **LuaJIT** スクリプトエンジンを搭載しています。タイムライン編集、クリップ配置、キーフレームアニメーション、独自のインタラクティブなドッキングツールパネルの構築をすべてスクリプトから自動化できます。

スクリプトディレクトリの配置

スクリプトやアイコンは、ユーザーの設定ディレクトリから自動的に検出・ロードされます:

ディレクトリ	役割 / 実行タイミング	概要
`~/.config/minicut/autorun/`	起動時自動登録	エディタ起動時に自動実行され、メニューバー下の水平「ドック (Dock)」にアイコンボタンを生成します。
`~/.config/minicut/scripts/`	メニュクリック時実行	メインメニューバーの **Scripts (スクリプト)** メニューに登録され、クリック時にパネルを開いて実行します。
`~/.config/minicut/icons/`	ドック用アイコン画像	スクリプト名と同名の 32x32 PNG アイコン画像（例: `quick_placement.lua` 用の `quick_placement.png`）。

2. プラグインスクリプトの基本構造

minicut のスクリプトは直感的なコールバック構造を持ちます。タイムラインやクリップのロジック関数を記述し、`plugin:init_panel()` メソッド内で Dear ImGui の UI コントロールを登録します。

```
-- ==========================================================================
-- 基本スクリプト例: my_tool.lua
-- ==========================================================================

local target_track = 0
local clip_scale = 1.25

-- タイムライン & クリップ操作ロジック
function execute_placement()
    minicut.push_undo()
    local path = "/path/to/media.mp4"
```

```
    local asset_id = project.import_video(path)
    local clip_id = timeline.place_video(target_track, asset_id,
timeline.get_playhead(), 5.0)
    if clip_id ~= "" then
        clip.set_scale(clip_id, clip_scale)
        timeline.add_keyframe(clip_id, "opacity", 0.0, 0.0)
        timeline.add_keyframe(clip_id, "opacity", 1.0, 1.0)
        minicut.set_status("トラック " .. target_track .. " にメディアを配置しました")
    end
end

-- GUI パネル初期化
function plugin:init_panel()
    gui.text("メディア配置ツール")
    gui.separator()
    target_track = gui.slider_int("対象トラック", target_track, 0, 4)
    clip_scale = gui.slider_float("初期スケール", clip_scale, 0.1, 5.0)

    if gui.button("再生ヘッド位置に配置", execute_placement) then
        -- インライン処理も可能
    end
end
```

3. コアモジュールリファレンス

3.1. プロジェクト & アセット操作 (`project` / Global)

関数	戻り値	説明
`project_save([path])` `project.save([path])`	`boolean, string`	プロジェクトを `.minicut` JSON ファイルとして保存します。
`project_load(path)` `project.load(path)`	`boolean, string`	指定パスからプロジェクトを読み込み、タイムラインを再構築します。
`project.get_name()`	`string`	現在のプロジェクト名を取得します。
`project.set_name(name)`	`void`	現在のプロジェクト名を設定します。
`project.import_video(path)`	`string (asset_id)`	動画ファイルをアセットプールにインポートします。
`project.import_audio(path)`	`string (asset_id)`	音声ファイルをアセットプールにインポートします。
`project.import_image(path, [dur])`	`string (asset_id)`	静止画像をアセットプールにインポートします（初期表示時間: 5 秒）。
`assets.create_text(text, [opts])`	`string (asset_id)`	FreeType を用いて高品質テキストアセットを生成します。

3.2. ネットワーク通信ライブラリ (net)

関数	戻り値	説明
net.get(url)	table	HTTP GET リクエストを送信します。{{status, body, ok, headers}} を返します。
net.post(url, body)	table	HTTP POST リクエストを文字列または JSON で送信します。
net.download(url, dest)	boolean	リモートファイルをローカルディスクに直接ダウンロード保存します。
net.upload(url, path)	table	ローカルファイルをリモートサーバへアップロードします。
net.request(options)	table	カスタム HTTP リクエスト（メソッド、ヘッダ、タイムアウト指定）。

3.3. タイムライン & トラック管理 (timeline)

関数	戻り値	説明
timeline.get_playhead()	number	現在の再生ヘッド位置（秒）を取得します。
timeline.set_playhead(t)	void	再生ヘッドを指定時刻（秒）へ移動します。
timeline.get_duration()	number	タイムライン全体の総再生時間を取得します。
timeline.get_track_count()	integer	全トラック数を取得します。
timeline.add_track(type, [name])	integer	新規トラック（"video" または "audio"）を追加します。
timeline.place_video(track, id, start, dur)	string	動画クリップを配置し、クリップ ID を返します。
timeline.place_audio(track, id, start, dur)	string	音声クリップを配置し、クリップ ID を返します。
timeline.place_image(track, id, start, dur)	string	画像クリップを配置し、クリップ ID を返します。

関数	戻り値	説明
`timeline.place_text_clip(track, text, start, dur)`	`string`	テキストクリップを生成して配置します。
`timeline.delete_clip(clip_id)`	`boolean`	クリップおよび関連キーフレームを削除します。
`timeline.split_clip(clip_id, [t])`	`boolean`	指定時刻でクリップを分割します（省略時は再生ヘッド位置）。
`timeline.select_clips(ids_table)`	`void`	複数クリップを一括選択します。
`timeline.get_selected_clips()`	`table`	選択中クリップ ID の配列を取得します。
`timeline.clear_selection()`	`void`	クリップの選択をすべて解除します。
`timeline.move_track_up(idx)`	`void`	トラックの並び順を 1 つ上に移動します。
`timeline.move_track_down(idx)`	`void`	トラックの並び順を 1 つ下に移動します。
`timeline.set_track_name(idx, name)`	`void`	トラック名を変更します。
`timeline.get_track_name(idx)`	`string`	トラック名を取得します。
`timeline.add_keyframe(id, prop, t, val)`	`void`	アニメーションキーフレームを追加します（opacity, volume 等）。
`timeline.remove_keyframe(id, prop, t)`	`void`	指定時刻のキーフレームを削除します。
`timeline.evaluate_keyframe(id, prop, t, [def])`	`number`	線形補間されたパラメータ値を取得します。

3.4. クリップ変形 & 属性操作 (`clip`)

関数	戻り値	説明
`clip.get_position(clip_id)`	`number, number`	キャンバス中央を基準とする (x, y) 座標を取得します。

関数	戻り値	説明
`clip.set_position(clip_id, x, y)`	`void`	クリップの中心座標を設定します。
`clip.get_scale(clip_id)`	`number`	スケール倍率（1.0 = 100%）を取得します。
`clip.set_scale(clip_id, s)`	`void`	スケール倍率を設定します。
`clip.get_rotation(clip_id)`	`number`	回転角度（度数法）を取得します。
`clip.set_rotation(clip_id, r)`	`void`	回転角度を設定します。
`clip.get_opacity(clip_id)`	`number`	不透明度 [0.0, 1.0] を取得します。
`clip.set_opacity(clip_id, o)`	`void`	不透明度を設定します。
`clip.get_volume(clip_id)`	`number`	音量倍率 [0.0, 2.0] を取得します。
`clip.set_volume(clip_id, v)`	`void`	音量倍率を設定します。
`clip.get_time(clip_id)`	`number, number, number`	(開始時刻, 長さ, オフセット) を取得します。
`clip.set_time(clip_id, start, dur, [off])`	`void`	タイムライン上の配置タイミングとオフセットを設定します。
`clip.get_info(clip_id)`	`table`	クリップの詳細プロパティ辞書を取得します。

3.5. GUI UI 要素 (gui)

関数	戻り値	説明
`gui.text(text)`	`void`	テキストを表示します。
`gui.text_colored(r, g, b, a, text)`	`void`	RGBA 色付きテキストを表示します。
`gui.button(label, [callback])`	`boolean`	プッシュボタンを表示し、クリック時にコールバックを実行します。
`gui.slider_float(label, val, min, max, [cb])`	`number`	浮動小数点スライダーを表示します。

関数	戻り値	説明
`gui.slider_int(label, val, min, max, [cb])`	`integer`	整数スライダーを表示します。
`gui.drag_float(label, val, [spd, min, max, cb])`	`number`	数値ドラッグボックスを表示します。
`gui.input_text(label, text, [cb])`	`string`	1 行テキスト入力欄を表示します。
`gui.checkbox(label, state, [cb])`	`boolean`	チェックボックスを表示します。
`gui.same_line()`	`void`	次のウィジェットを同じ行に配置します。
`gui.separator()`	`void`	水平区切り線を描画します。
`gui.file_dialog(options)`	`void`	ユーザー設定に基づきファイルダイアログを開きます。

3.6. エンジンユーティリティ (`minicut`)

関数	戻り値	説明
`minicut.push_undo()`	`void`	元に戻す（Undo）用のスナップショットを記録します。
`minicut.set_status(msg, [dur])`	`void`	ステータスバーに一時的な通知メッセージを表示します。
`minicut.log(msg)`	`void`	コンソールへデバッグログを出力します。